

Laporan Sementara Scikit-Learn

Nama : Albie Bahirshah Widyanto
NIM : 234308060
Kelas : TKA-6C
Github : <https://github.com/Albie788>

A. Pendahuluan

Perkembangan interaksi manusia dan komputer (*Human-Computer Interaction*) telah mengalami transformasi besar seiring dengan kemajuan teknologi *Computer Vision*. Kamera tidak lagi hanya digunakan sebagai alat perekam visual, tetapi telah berkembang menjadi sensor cerdas yang mampu memahami gerakan dan gestur manusia secara *real-time*. Salah satu tantangan utama dalam bidang ini adalah bagaimana mengekstraksi informasi koordinat tubuh yang presisi dengan beban komputasi yang ringan agar dapat berjalan pada perangkat standar. Dalam praktikum ini, teknologi yang digunakan adalah framework MediaPipe yang dikembangkan oleh Google. MediaPipe menawarkan solusi *open-source* untuk pelacakan tangan dan tubuh yang bekerja dengan memetakan titik-titik koordinat atau *landmarks* (Lugaresi et al., 2019). Penggunaan MediaPipe dianggap lebih efisien dibandingkan metode berbasis *Deep Learning* konvensional karena tidak memerlukan sumber daya GPU yang sangat besar untuk tahap inferensi awal. Untuk mengklasifikasikan data koordinat yang dihasilkan oleh MediaPipe, digunakan algoritma Random Forest Classifier. Algoritma ini merupakan salah satu metode *ensemble learning* yang bekerja dengan membangun banyak pohon keputusan untuk meningkatkan akurasi klasifikasi dan mencegah *overfitting* (Breiman, 2001). Implementasi praktikum ini difokuskan pada pengenalan gestur tangan untuk mendukung Sistem Isyarat Bahasa Indonesia (SIBI). Hal ini menjadi krusial karena teknologi pengenalan isyarat dapat menjadi jembatan komunikasi bagi penyandang disabilitas rungu dan wicara. Selain itu, praktikum ini juga mencakup deteksi pose tubuh untuk membedakan posisi berdiri dan duduk menggunakan MediaPipe Pose. Melalui integrasi antara pustaka OpenCV untuk pengolahan citra dan Scikit-Learn untuk pemodelan mesin, sistem diharapkan mampu memberikan prediksi label yang akurat berdasarkan ekstraksi fitur koordinat yang telah dinormalisasi.

B. Tujuan

- Memahami penerapan Computer Vision dalam Human-Computer Interaction (HCI) untuk pengenalan gestur tangan dan pose tubuh secara real-time.
- Mengimplementasikan framework MediaPipe dan algoritma Random Forest Classifier dalam proses ekstraksi fitur koordinat serta klasifikasi data gestur.
- Membangun sistem pengenalan gestur SIBI dan deteksi pose (duduk dan berdiri) **berbasis** integrasi OpenCV dan Scikit-Learn.

C. Manfaat

- Meningkatkan keterampilan mahasiswa dalam pengolahan citra digital dan machine learning, mulai dari akuisisi data hingga tahap klasifikasi.
- Memberikan pengalaman integrasi berbagai pustaka pemrograman dalam membangun sistem cerdas berbasis visi komputer.
- Mendukung pengembangan teknologi asistif, khususnya sistem pengenalan bahasa isyarat untuk membantu komunikasi penyandang disabilitas rungu dan wicara.

D. Hasil Percobaan

1. Machine Learning Gesture Detection

```
import os
import cv2

DATA_DIR = 'E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\DATA'
if not os.path.exists(DATA_DIR):
    os.makedirs(DATA_DIR)

number_of_classes = 2
dataset_size = 100
cap = cv2.VideoCapture(0)

for j in range(number_of_classes):
    if not os.path.exists(os.path.join(DATA_DIR, str(j))):
        os.makedirs(os.path.join(DATA_DIR, str(j)))

    print('Collecting data for class {}'.format(j))
    while True:
        ret, frame = cap.read()
        cv2.putText(frame, 'Ready? Press "Q" !', (100, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 3)
        cv2.imshow('frame', frame)
        if cv2.waitKey(15) == ord('q'):
            break

    counter = 0
    while counter < dataset_size:
        ret, frame = cap.read()
        cv2.imshow('frame', frame)
        cv2.waitKey(10)
        cv2.imwrite(os.path.join(DATA_DIR, str(j), '{}.jpg'.format(counter)), frame)
        counter += 1

cap.release()
cv2.destroyAllWindows()
```

Program ini menggunakan OpenCV dan OS untuk mengumpulkan dataset gambar secara otomatis melalui *webcam*. Alurnya dimulai dengan menyiapkan folder untuk 2 kategori kelas, lalu membuka kamera dan menunggu pengguna menekan tombol 'q' sebagai pemicu pengambilan data. Setelah dipicu, sistem secara otomatis menangkap dan menyimpan 100 gambar per kelas ke dalam folder tujuan dengan penamaan file yang berurutan. Terakhir, program akan menutup akses kamera dan membersihkan jendela tampilan setelah seluruh proses selesai.

2. Ekstraksi data dari datasheet

```
import os
import pickle
import mediapipe as mp
import cv2

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(static_image_mode=True, max_num_hands=1)

DATA_DIR = 'E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\DATA'
data = []
labels = []

for dir_ in os.listdir(DATA_DIR):
    for img_path in os.listdir(os.path.join(DATA_DIR, dir_)):
        data_aux = []
        x_ = []
        y_ = []
        img = cv2.imread(os.path.join(DATA_DIR, dir_, img_path))
        img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        results = hands.process(img_rgb)

        if results.multi_hand_landmarks:
            for hand_landmarks in results.multi_hand_landmarks:
                for i in range(len(hand_landmarks.landmark)):
                    x = hand_landmarks.landmark[i].x
                    y = hand_landmarks.landmark[i].y
                    x_.append(x)
                    y_.append(y)

                for i in range(len(hand_landmarks.landmark)):
                    x = hand_landmarks.landmark[i].x
                    y = hand_landmarks.landmark[i].y
                    data_aux.append(x - min(x_))
                    data_aux.append(y - min(y_))

            data.append(data_aux)
            labels.append(dir_)

f = open('E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\data.pickle', 'wb')
pickle.dump({'data': data, 'labels': labels}, f)
f.close()
```

Rangkaian program ini berfungsi untuk mengumpulkan dataset gambar tangan melalui *webcam* menggunakan OpenCV dan mengekstraksi fiturnya menggunakan MediaPipe. Sistem menangkap 100 gambar untuk tiap kategori kelas yang kemudian diproses untuk mengambil koordinat titik tangan (*landmarks*). Data koordinat tersebut dinormalisasi dan disimpan bersama labelnya ke dalam file pickle, sehingga siap digunakan sebagai input untuk pelatihan model *machine learning*.

3. Pelatihan dan pengujian

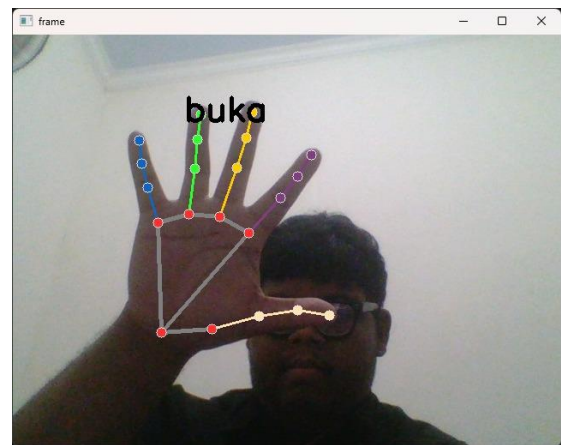
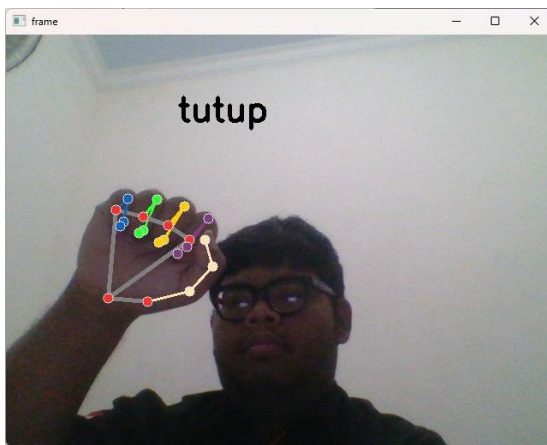
```
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

data_dict = pickle.load(open('E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\data.pickle', 'rb'))
data = np.asarray(data_dict['data'])
labels = np.asarray(data_dict['labels'])
x_train, x_test, y_train, y_test = train_test_split(data, labels, test_size=0.1, shuffle=True, stratify=labels)
model = RandomForestClassifier()
model.fit(x_train, y_train)
y_predict = model.predict(x_test)
score = accuracy_score(y_predict, y_test)
print('{}% of samples were classified correctly !'.format(score * 100))

f = open('E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\model.p', 'wb')
pickle.dump({'model.p': model}, f)
f.close()
```

Rangkaian program ini merupakan sistem klasifikasi citra yang dimulai dengan pengumpulan data sebanyak 100 gambar untuk dua kategori berbeda menggunakan OpenCV. Data gambar tersebut kemudian diproses menggunakan MediaPipe untuk mengekstraksi koordinat titik tangan (*hand landmarks*) yang dinormalisasi dan disimpan ke dalam file pickle. Pada tahap akhir, data fitur tersebut digunakan untuk melatih model Random Forest Classifier yang akan mengukur tingkat akurasi prediksi sebelum akhirnya disimpan sebagai file model siap pakai.

4. Implementasi



```
import ...

model_dict = pickle.load(open('E:\Tugas-Tugas masuk sini titikduav\Praktikum Kontrol Cerdas\Pertemuan 3\model.p'))
model = model_dict['model.p']
cap = cv2.VideoCapture(0)
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
hands = mp_hands.Hands(max_num_hands=1)

labels_dict = {0: 'buka', 1: 'tutup'}

while True:
    data_aux = []
    x_ = []
    y_ = []
    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(frame_rgb)

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                frame,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style()
            )

            for hand_landmarks in results.multi_hand_landmarks:
                for i in range(len(hand_landmarks.landmark)):
                    x = hand_landmarks.landmark[i].x
                    y = hand_landmarks.landmark[i].y
                    x_.append(x)
                    y_.append(y)
                    data_aux.append(x - min(x_))
                    data_aux.append(y - min(y_))

            prediction = model.predict([np.asarray(data_aux)])
            predicted_character = labels_dict[int(prediction[0])]
            cv2.putText(frame, predicted_character, (200, 100), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 0, 0), 3)

    cv2.imshow('frame', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

5. Latihan soal A

```
import pickle
import cv2
import mediapipe as mp
import numpy as np
import string

# Load model SIBI
model_dict = pickle.load(open('E:\Tugas-Tugas masuk sini titikduay\Praktikum Kontrol Cerdas\Pertemuan 3\model_sibi.p', 'rb'))
model = model_dict['model_sibi.p']

cap = cv2.VideoCapture(0)
mp_hands = mp.solutions.hands
mp_drawing = mp.solutions.drawing_utils
mp_drawing_styles = mp.solutions.drawing_styles
hands = mp_hands.Hands(max_num_hands=1)

# Membuat dictionary 0-25 menjadi A-Z secara otomatis
labels_dict = {i: char for i, char in enumerate(string.ascii_uppercase)}

while True:
    data_aux = []
    x_ = []
    y_ = []

    ret, frame = cap.read()
    if not ret: break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = hands.process(frame_rgb)

    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                frame,
                hand_landmarks,
                mp_hands.HAND_CONNECTIONS,
                mp_drawing_styles.get_default_hand_landmarks_style(),
                mp_drawing_styles.get_default_hand_connections_style()
            )

            for i in range(len(hand_landmarks.landmark)):
                x = hand_landmarks.landmark[i].x
                y = hand_landmarks.landmark[i].y
                x_.append(x)
                y_.append(y)
                data_aux.append(x - min(x_))
                data_aux.append(y - min(y_))

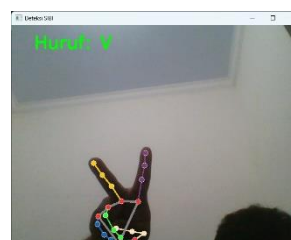
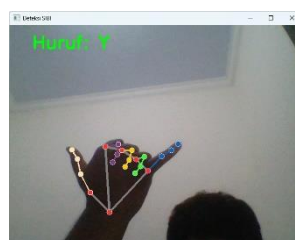
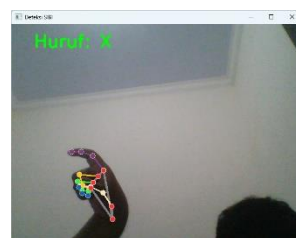
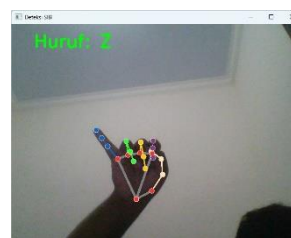
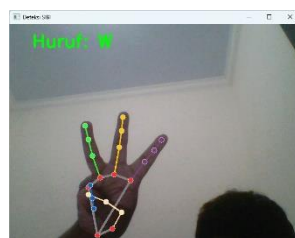
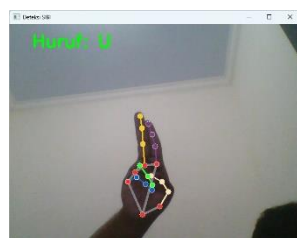
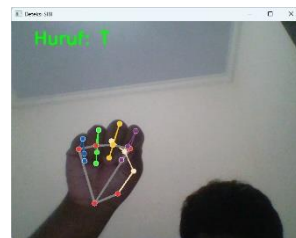
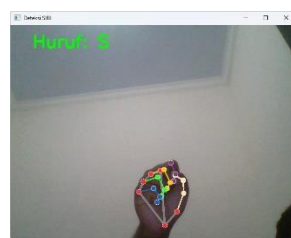
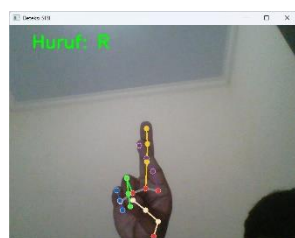
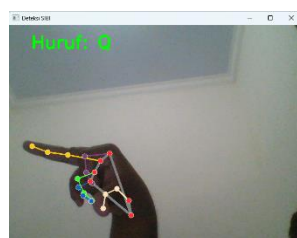
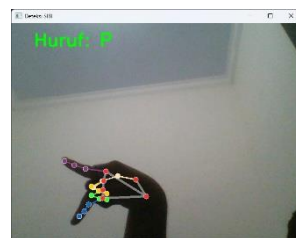
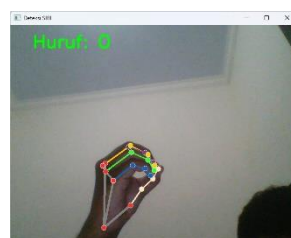
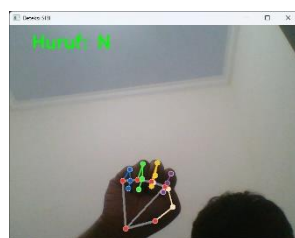
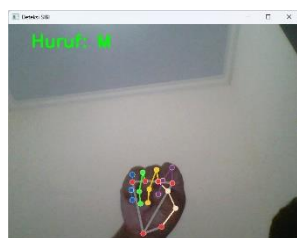
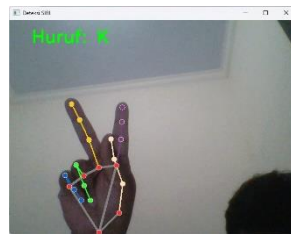
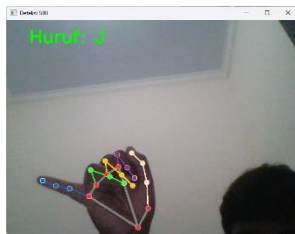
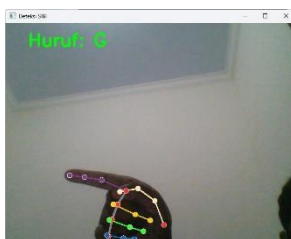
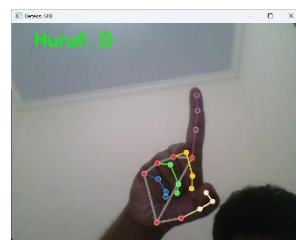
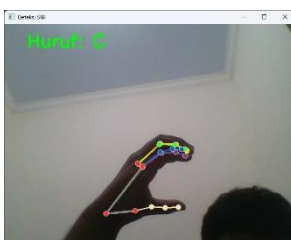
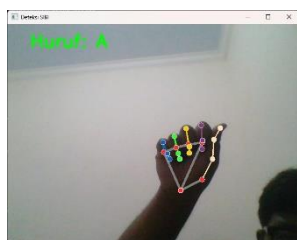
            # Prediksi huruf SIBI
            prediction = model.predict([np.asarray(data_aux)])
            predicted_character = labels_dict[int(prediction[0])]

            cv2.putText(frame, f'Huruf: {predicted_character}', (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0, 255, 0), 3)

    cv2.imshow('Deteksi SIBI', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```



DAFTAR PUSTAKA

Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.

Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>

Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., ... & Varoquaux, G. (2013). API design for machine learning software: experiences from the scikit-learn project. *arXiv preprint arXiv:1309.0238*.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., ... & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. *arXiv preprint arXiv:1906.08172*.

Zhang, F., Bazarevsky, V., Vakunov, A., Wu, H., Pan, S., Gu, J., ... & Grundmann, M. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. *arXiv preprint arXiv:2006.10214*.